

UNIVERSIDAD AMERICANA



Documentación Ejercicios Grafos.

Integrantes:

David Sánchez.

Alicia Estrada.

Franco Aguilera.

Sara Zambrana.

Andrea Duarte.

Profesora: Silvia Ticay.

Fecha: 16/06/25.

Ejercicio 1

grafos.py

```
from collections import defaultdict
```

Se utiliza defaultdict, el cual es un diccionario que automáticamente inicializa valores por defecto para claves nuevas, evitando errores al acceder a claves que aún no existen.

```
class Grafo:
    def __init__(self, es_dirigido=False):
        # Diccionario donde las claves son vértices y los valores son listas de tuplas (vecino, peso)
        self.grafo = defaultdict(list)
        self.es_dirigido = es_dirigido # Controla si el grafo es dirigido o no
```

Se define la clase principal que representa el grafo. Y se utiliza un diccionario (defaultdict) para almacenar listas de adyacencia. El parámetro “es_dirigido” indica si el grafo es dirigido (True) o no (False).

```
def agregar_vertice(self, vertice):
    # Agrega un vértice si no existe. Si ya existe, no hace nada.
    if vertice not in self.grafo:
        self.grafo[vertice] = []
```

Se verifica si el vértice ya existe en el grafo, y si no existe, se crea con una lista vacía con “self.grafo[vertice] = []”.

```
def agregar_arista(self, u, v, peso=1):
    # Agrega una arista entre u y v. Si los vértices no existen, se crean automáticamente.
    # Si es no dirigido, agrega la arista en ambos sentidos.
    self.agregar_vertice(u)
    self.agregar_vertice(v)
    if (v, peso) not in self.grafo[u]:
        self.grafo[u].append((v, peso))
    if not self.es_dirigido:
        if (u, peso) not in self.grafo[v]:
            self.grafo[v].append((u, peso))
```

self.agregar_vertice(u) y self.agregar_vertice(v): Asegura que los dos vértices existan. Si no, los crea.

if (v, peso) not in self.grafo[u]: Revisa si ya existe esa conexión desde u hacia v con ese peso.

self.grafo[u].append((v, peso)): Si no existía, la agrega.

if not self.es_dirigido: Si el grafo no es dirigido, repite el paso anterior pero al revés (de v hacia u).

```
def obtener_vecinos(self, vertice):
    # Devuelve solo los nombres de los vecinos (no los pesos). Si el vértice no existe, devuelve lista vacía.
    return [vecino for vecino, _ in self.grafo.get(vertice, [])]
```

self.grafo.get(vertice, []): Busca el vértice. Si no existe, devuelve una lista vacía.

[vecino for vecino, _ in ...]: Recorre cada (vecino, peso) y se queda solo con el nombre del vecino.

```
def existe_arista(self, u, v):  
    # Retorna True si existe una arista de u a v (verifica solo dirección u -> v)  
    return any(vecino == v for vecino, _ in self.grafo.get(u, []))
```

self.grafo.get(u, []): Obtiene los vecinos de u. Si no existe, devuelve [].

any(vecino == v for vecino, _ in ...): Revisa si v está en la lista de vecinos de u.

Devuelve True si existe una arista de u a v, y False si no.

```
def verificar_vecinos(grafo, vertice):  
  
    # Devuelve e imprime los vecinos de un vértice. Informa si el vértice no existe.  
  
    vecinos = grafo.obtener_vecinos(vertice)  
    if vecinos:  
        print(f"Vecinos de '{vertice}': {vecinos}")  
        return vecinos  
    else:  
        if vertice in grafo.grafo:  
            print(f"El vértice '{vertice}' existe pero no tiene vecinos.")  
            return []  
        else:  
            print(f"El vértice '{vertice}' NO existe en el grafo.")  
            return None
```

Usa el método obtener_vecinos() de la clase Grafo para consultar a qué vértices está conectado el vértice dado.

Si tiene vecinos, los imprime.

Si no tiene vecinos:

- Verifica si el vértice existe en el grafo (pero no tiene conexiones).
- Si no existe, se avisa al usuario.

```
def verificar_arista(grafo, u, v):  
  
    # Indica si existe una arista entre u y v.  
  
    existe = grafo.existe_arista(u, v)  
    if existe:  
        print(f"Sí existe una arista entre '{u}' y '{v}'.")  
    else:  
        print(f"No existe una arista entre '{u}' y '{v}'.")  
    return existe
```

Llama al método existe_arista() para ver si hay una conexión desde u hasta v.

```
def mostrar_grafo(grafo):
    # Muestra toda la estructura del grafo: vértices y sus vecinos con peso.
    print("\n--- Estructura completa del grafo ---")
    for vertice, vecinos in grafo.grafo.items():
        lista = [f"{vecino} (peso = {peso})" for vecino, peso in vecinos]
        print(f"{vertice} -> {' '.join(lista)}")
    print("-----")
```

Se recorren todos los vértices del grafo, por cada vértice, muestra sus conexiones (vecinos) con los respectivos pesos.

```
def mostrar_grafo_adyacencia(grafo):
    print("\033[94m\n--- Lista de adyacencia del grafo ---\033[0m") # Azul
    for vertice, vecinos in grafo.grafo.items():
        vecinos_str = ", ".join(
            f"\033[92m{vecino}\033[0m (peso={peso})" for vecino, peso in vecinos
        )
        print(f" \033[96m{vertice}\033[0m → {vecinos_str}")
    print("\033[94m-----\033[0m")
```

Se imprime con colores.

main.py

```
from grafo import Grafo
import os

from grafo import verificar_vecinos, verificar_arista, mostrar_grafo, mostrar_grafo_adyacencia

def limpiar_pantalla():
    """Limpia la pantalla de la consola."""
    input("Presione Enter para continuar...")
    os.system('cls' if os.name == 'nt' else 'clear')
```

Se importa la clase Grafo desde el módulo grafo, junto con la librería os que permite ejecutar comandos del sistema operativo para limpiar la pantalla.

```
# Grafo no dirigido
def menu_grafos():

    grafo = Grafo(es_dirigido=False)
    while True:

        # Mostrar el menú de opciones
        print("\n--- MENÚ GRAFOS ---")
        print("1. Agregar vértice")
        print("2. Agregar arista")
        print("3. Consultar vecinos de un vértice")
        print("4. Verificar existencia de arista")
        print("5. Mostrar estructura completa del grafo")
        print("6. Salir")
        print("-----")
        try :
            opcion = int(input("Elige una opción: ").strip())
        except ValueError:
            print("Entrada inválida. Por favor, ingresa un número.")
            limpiar_pantalla()
            continue
```

Se crea un objeto grafo de la clase Grafo con el parámetro `es_dirigido=False`, lo que significa que el grafo será no dirigido.

Se validan los datos de entrada con `try`, en caso que el usuario ingrese otra opción se pasa al `except`.

```
match opcion:
    case 1:
        vertice = input("Nombre del vértice: ").strip()
        grafo.agregar_vertice(vertice)
        print(f"Vértice '{vertice}' agregado.")
        limpiar_pantalla()
    case 2:
        u = input("Vértice de inicio: ").strip()
        v = input("Vértice de destino: ").strip()
        try:
            peso = int(input("Peso (opcional, por defecto 1): ").strip() or "1")
        except ValueError:
            print("Peso inválido, se usará 1.")
            peso = 1
        grafo.agregar_arista(u, v, peso)
        if grafo.es_dirigido:
            print(f"Arista dirigida '{u}' -> '{v}' (peso={peso}) agregada.")
        else:
            print(f"Arista no dirigida '{u}' <-> '{v}' (peso={peso}) agregada.")
        limpiar_pantalla()
    case 3:
        vertice = input("Vértice a consultar: ").strip()
        verificar_vecinos(grafo, vertice)
        limpiar_pantalla()
    case 4:
        u = input("Vértice de inicio: ").strip() # Retorna una copia de la cadena con los espacios en blanco al inicio y al final eliminados
        v = input("Vértice de destino: ").strip() # Return a copy of the string with leading and trailing whitespace removed
        verificar_arista(grafo, u, v)
        limpiar_pantalla()
    case 5:
        mostrar_grafo_adyacencia(grafo)
        limpiar_pantalla()
    case 6:
        print("¡Hasta luego!")
        break
    case _:
        print("Opción inválida. Intenta de nuevo.")
        limpiar_pantalla()
```

Case 1: Se solicita al usuario que ingrese el nombre del nuevo vértice, y luego, se agrega ese vértice al grafo con el método `agregar_vertice`.

Case 2: Agrega una arista entre dos vértices. Se piden los nombres de los vértices de inicio y destino.

Case 3: Se solicita el nombre de un vértice para consultar sus vecinos. La función `verificar_vecinos` imprime los vecinos de ese vértice, si existen, y luego se limpia la pantalla.

Case 4: Permite verificar si existe una arista entre dos vértices dados. Se piden ambos vértices, se llama a la función `verificar_arista` para informar si la arista existe o no.

Case 5: Se muestra la estructura completa del grafo en forma de lista de adyacencia mediante la función `mostrar_grafo_adyacencia`.

```
        case _:
            print("Opción inválida. Intenta de nuevo.")
            limpiar_pantalla()

        print("\n") # Espacio entre iteraciones del menú
# Importar las funciones de verificación y mostrar del grafo
if __name__ == '__main__':
    menu_grafos()
```

Ejercicio 2

```
from collections import defaultdict, deque

class Grafo:
    """
    Clase Grafo: Estructura de datos para representar grafos dirigidos o no dirigidos usando listas de adyacencia.
    Permite agregar vértices y aristas, consultar vecinos, verificar existencia de aristas y realizar recorridos BFS y DFS.
    """
    def __init__(self, es_dirigido=False):
        self.grafo = defaultdict(list)
        self.es_dirigido = es_dirigido
```

Constructor de la clase. Crea un diccionario con listas como valores para almacenar los vértices y sus conexiones. También guarda si el grafo es dirigido o no.

```
def agregar_vertice(self, vertice):
    if vertice not in self.grafo:
        self.grafo[vertice] = []

def agregar_arista(self, u, v, peso=1):
    self.agregar_vertice(u)
    self.agregar_vertice(v)
    if (v, peso) not in self.grafo[u]:
        self.grafo[u].append((v, peso))
    if not self.es_dirigido:
        if (u, peso) not in self.grafo[v]:
            self.grafo[v].append((u, peso))
```

El método “agregar_vertice” añade un vértice al grafo si aún no existe.

El método “agregar_arista” crea una conexión (arista) entre dos vértices u y v, con un peso. Si el grafo no es dirigido, también agrega la arista en sentido inverso.

```
def obtener_vecinos(self, vertice):
    return [vecino for vecino, _ in self.grafo.get(vertice, [])]
```

El método obtener_vecinos devuelve una lista solo con los nombres de los vecinos.

- self.grafo.get(vertice, []) obtiene la lista de tuplas (vecino, peso) del vértice.
- El for vecino, _ extrae solo el nombre del vecino, ignorando el peso (_).
- Si el vértice no existe, devuelve una lista vacía.

```
def existe_arista(self, u, v):
    return any(vecino == v for vecino, _ in self.grafo.get(u, []))
```

Se busca en la lista de vecinos de u si alguno tiene como nombre v.

Devuelve True si encuentra al menos uno (con any), de lo contrario False.

```

def mostrar_adyacencia(self, grafo):
    print("\033[94m\n--- Lista de adyacencia del grafo ---\033[0m") # Azul
    for vertice, vecinos in grafo.grafo.items():
        vecinos_str = ", ".join(
            f"\033[92m{vecino}\033[0m (peso={peso})" for vecino, peso in vecinos
        )
        print(f" \033[96m{vertice}\033[0m → {vecinos_str}")
    print("\033[94m-----\033[0m")
    print("\033[92mGrafo creado exitosamente.\033[0m") # Verde

```

Muestra en pantalla la representación del grafo como lista de adyacencia.

```

def bfs(self, inicio):
    """
    Recorrido en amplitud (BFS) usando una cola. Devuelve la lista de vértices en el orden en que se visitan.
    """
    if inicio not in self.grafo:
        print(f"El vértice '{inicio}' no existe en el grafo.")
        return []
    visitados = set()
    cola = deque()
    recorrido = []
    cola.append(inicio)
    visitados.add(inicio)
    while cola:
        actual = cola.popleft()
        recorrido.append(actual)
        for vecino in self.obtener_vecinos(actual):
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
    return recorrido

```

Verifica que el vértice de inicio exista, luego usa una cola para mantener el orden de exploración, usa un set de visitados para evitar ciclos o repeticiones, va sacando vértices de la cola, los agrega al recorrido y pone sus vecinos no visitados al final de la cola, y por último, devuelve la lista de vértices en el orden en que fueron visitados.

```

def dfs(self, inicio):
    """
    Recorrido en profundidad (DFS) usando recursividad. Devuelve la lista de vértices en el orden en que se visitan.
    """
    if inicio not in self.grafo:
        print(f"El vértice '{inicio}' no existe en el grafo.")
        return []
    visitados = set()
    recorrido = []
    def _dfs_rec(v):
        visitados.add(v)
        recorrido.append(v)
        for vecino in self.obtener_vecinos(v):
            if vecino not in visitados:
                _dfs_rec(vecino)
    _dfs_rec(inicio)
    return recorrido

```

Este método realiza un recorrido en profundidad desde un vértice inicial usando recursividad. Primero verifica si el vértice existe. Luego, mediante una función interna llamada `_dfs_rec`, visita el vértice actual, lo marca como visitado y recorre recursivamente a sus vecinos no visitados. Devuelve una lista con el orden en que fueron visitados los vértices.

```
def verificar_vecinos(grafo, vertice):
    vecinos = grafo.obtener_vecinos(vertice)
    if vecinos:
        print(f"Vecinos de '{vertice}': {vecinos}")
    else:
        if vertice in grafo.grafo:
            print(f"El vértice '{vertice}' existe pero no tiene vecinos.")
        else:
            print(f"El vértice '{vertice}' NO existe en el grafo.")
```

Esta función recibe un vértice y muestra en pantalla todos sus vecinos directos, es decir, los vértices conectados a él. Si el vértice no tiene vecinos, indica si está aislado o si no existe en el grafo. Es útil para explorar la estructura del grafo de forma interactiva.

```
def verificar_arista(grafo, u, v):
    existe = grafo.existe_arista(u, v)
    if existe:
        print(f"Sí existe una arista entre '{u}' y '{v}'.")
    else:
        print(f"No existe una arista entre '{u}' y '{v}'.")
```

Esta función verifica si existe una conexión directa (una arista) entre dos vértices dados. Utiliza el método `existe_arista` del grafo y muestra un mensaje indicando si la arista está presente o no. Es útil para comprobar relaciones específicas dentro del grafo.

```
from modulo import Grafo, verificar_vecinos, verificar_arista
class Funciones:
    def opcion_agregar_vertice(grafo):
        vertice = input("Nombre del vértice: ").strip()
        grafo.agregar_vertice(vertice)
        print(f"Vértice '{vertice}' agregado.")

    def opcion_agregar_arista(grafo):
        u = input("Vértice de inicio: ").strip()
        v = input("Vértice de destino: ").strip()
        try:
            peso = int(input("Peso (opcional, por defecto 1): ").strip() or "1")
        except ValueError:
            print("Peso inválido, se usará 1.")
            peso = 1
        grafo.agregar_arista(u, v, peso)
        if grafo.es_dirigido:
            print(f"Arista '{u}' -> '{v}' agregada (peso={peso}).")
        else:
            print(f"Arista '{u}' <-> '{v}' agregada (peso={peso}).")
```

El método "opcion_agregar_vertice" solicita al usuario el nombre de un vértice y lo agrega al grafo. Se usa `strip()` para eliminar espacios innecesarios. Luego se muestra un mensaje confirmando la acción.

El método “opcion_agregar_arista” agrega una arista entre dos vértices introducidos por el usuario. El peso puede ser ingresado manualmente o usar el valor por defecto. Si el grafo es dirigido, se muestra la dirección correcta de la arista.

```
def opcion_consultar_vecinos(grafo):
    vertice = input("Vértice a consultar: ").strip()
    vecinos = grafo.obtener_vecinos(vertice)
    if vecinos:
        print(f"Vecinos de '{vertice}': {vecinos}")
    else:
        if vertice in grafo.grafo:
            print(f"El vértice '{vertice}' existe pero no tiene vecinos.")
        else:
            print(f"El vértice '{vertice}' NO existe en el grafo.")

def opcion_verificar_arista(grafo):
    u = input("Vértice de inicio: ").strip()
    v = input("Vértice de destino: ").strip()
    if grafo.existe_arista(u, v):
        print(f"Sí existe una arista entre '{u}' y '{v}'.")
    else:
        print(f"No existe una arista entre '{u}' y '{v}'.")
```

opcion_consultar_vecinos: Permite consultar los vecinos de un vértice. Si no tiene vecinos o no existe, se informa adecuadamente al usuario con un mensaje específico.

opcion_verificar_arista: Verifica si existe una arista entre dos vértices específicos. Usa el método existe_arista del grafo para confirmar la existencia.

```
def opcion_mostrar_adyacencia(grafo):
    grafo.mostrar_adyacencia()

def opcion_bfs(grafo):
    inicio = input("Vértice inicial para BFS: ").strip()
    resultado = grafo.bfs(inicio)
    if resultado:
        print(f"Recorrido BFS desde '{inicio}': {resultado}")

def opcion_dfs(grafo):
    inicio = input("Vértice inicial para DFS: ").strip()
    resultado = grafo.dfs(inicio)
    if resultado:
        print(f"Recorrido DFS desde '{inicio}': {resultado}")

def opcion_desconexo(grafo):
    print("Agregando vértice 'F' (desconexo)...")
    grafo.agregar_vertice('F')
    grafo.mostrar_adyacencia()
    inicio = input("Vértice inicial para BFS/DFS: ").strip()
    print(f"BFS: {grafo.bfs(inicio)}")
    print(f"DFS: {grafo.dfs(inicio)}")
```

Opcion_mostrar_adyacencia: Llama al método del grafo que muestra la lista de adyacencia completa, incluyendo vértices y sus conexiones con pesos.

Opcion_bfs: Ejecuta un recorrido en amplitud (BFS) desde un vértice dado. Se muestra el orden de los vértices visitados en el recorrido.

Opcion_dfs: Realiza un recorrido en profundidad (DFS) desde un vértice inicial indicado por el usuario, mostrando el orden de visita de los nodos.

Opcion_desconexo: Este bloque agrega un vértice aislado llamado 'F' para probar grafos no conexos. Luego permite realizar BFS y DFS desde cualquier vértice para observar el comportamiento.

main.py

```
from modulo import Grafo
import os
from funciones import Funciones as f

'''
Módulo principal para interactuar con el grafo.
Permite al usuario crear un grafo dirigido o no dirigido y realizar diversas operaciones sobre él.
'''

def limpiar_pantalla():
    """Limpia la pantalla de la terminal."""
    input("Presione Enter para continuar...")
    os.system('cls' if os.name == 'nt' else 'clear')

def menu_grafos():
    print("¿Deseas trabajar con un grafo dirigido? (s/n): ", end="")
    dirigido = input().strip().lower() == 's' # True si es dirigido, False si no lo es
    grafo = Grafo(es_dirigido=False) if not dirigido else Grafo(es_dirigido=True)
    print("\nGrafo creado!\n")

    while True:
        print("\n--- MENÚ GRAFOS ---")
        print("1. Agregar vértice")
        print("2. Agregar arista")
        print("3. Consultar vecinos de un vértice")
        print("4. Verificar existencia de arista")
        print("5. Mostrar lista de adyacencia")
        print("6. Recorrido BFS")
        print("7. Recorrido DFS")
        print("8. Prueba de recorrido en grafo desconexo")
        print("0. Salir")
        print("-----")
        try:
```

```

try:

    opcion = int(input("Elige una opción: "))
except ValueError:
    print("Entrada inválida. Por favor, ingresa un número.")
    continue
match opcion:
    case 1:
        f.opcion_agregar_vertice(grafo)
        limpiar_pantalla()
    case 2:
        f.opcion_agregar_arista(grafo)
        limpiar_pantalla()
    case 3:
        f.opcion_consultar_vecinos(grafo)
        limpiar_pantalla()
    case 4:
        f.opcion_verificar_arista(grafo)
        limpiar_pantalla()
    case 5:
        f.opcion_mostrar_adyacencia(grafo)
        limpiar_pantalla()
    case 6:
        f.opcion_bfs(grafo)
        limpiar_pantalla()
    case 7:
        f.opcion_dfs(grafo)
        limpiar_pantalla()
    case 8:
        f.opcion_desconexo(grafo)
        limpiar_pantalla()
    case 0:
        print("|Hasta luego!")
        break
    case _:
        print("Opción inválida. Intenta de nuevo.")

```

```

        case _:
            print("Opción inválida. Intenta de nuevo.")

if __name__ == "__main__":
    menu_grafos()

```

Ejercicio 3

grafo.py

```
class Grafo:
    """
    Clase principal de grafos del equipo.
    Permite grafos dirigidos o no, aristas con peso y recorridos BFS/DFS.
    Incluye es_conexo y encontrar_camino como pide el ejercicio.
    """
    def __init__(self, es_dirigido=False):
        self.grafo = defaultdict(list)
        self.es_dirigido = es_dirigido

    def agregar_vertice(self, vertice):
        if vertice not in self.grafo:
            self.grafo[vertice] = []

    def agregar_arista(self, u, v, peso=1):
        # Siempre añade los vértices primero (por si no existen)
        self.agregar_vertice(u)
        self.agregar_vertice(v)
        if (v, peso) not in self.grafo[u]:
            self.grafo[u].append((v, peso))
        if not self.es_dirigido:
            if (u, peso) not in self.grafo[v]:
                self.grafo[v].append((u, peso))

    def obtener_vecinos(self, vertice):
        return [vecino for vecino, _ in self.grafo.get(vertice, [])]

    def existe_arista(self, u, v):
        return any(vecino == v for vecino, _ in self.grafo.get(u, []))
```

`__init__(self, es_dirigido=False)`: Este método constructor inicializa el grafo como un diccionario de listas (`defaultdict(list)`), donde cada vértice tendrá su lista de vecinos.

`agregar_vertice(self, vertice)`: Agrega un vértice al grafo solo si no existe ya. Aunque se puede agregar manualmente, normalmente se hace automáticamente al agregar una arista entre vértices nuevos.

`agregar_arista(self, u, v, peso=1)`: Agrega una conexión entre dos vértices `u` y `v` con un peso opcional (por defecto 1).

- Si el grafo es dirigido, solo se agrega la arista de `u` a `v`.
- Si no es dirigido, también se agrega la arista de `v` a `u`, simulando doble vía. También se asegura que ambos vértices existan antes de agregar la arista.

obtener_vecinos(self, vertice): Devuelve una lista con los vecinos (vértices adyacentes) del vértice dado, ignorando los pesos.

existe_arista(self, u, v): Verifica si existe una arista desde u hasta v.

Solo se fija en los nombres de los vértices, ignorando el peso.

Esto es útil para confirmar si hay conexión directa entre dos vértices.

```
def mostrar_adyacencia(self):
    print("\n--- Lista de adyacencia del grafo ---")
    for vertice, vecinos in self.grafo.items():
        vecinos_str = ", ".join(f"{vecino}(peso={peso})" for vecino, peso in vecinos)
        print(f"  {vertice} -> {vecinos_str}")
    print("-----")

def bfs(self, inicio):
    if inicio not in self.grafo:
        print(f"El vértice '{inicio}' no existe en el grafo.")
        return []
    visitados = set()
    cola = deque([inicio])
    visitados.add(inicio)
    recorrido = []
    while cola:
        actual = cola.popleft()
        recorrido.append(actual)
        for vecino in self.obtener_vecinos(actual):
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)
    return recorrido
```

Método mostrar_adyacencia(self)

- El for vertice, vecinos in self.grafo.items(): recorre todos los vértices y sus listas de vecinos almacenados en self.grafo.
- En vecinos_str = ", ".join(f"{vecino}(peso={peso})" for vecino, peso in vecinos) construye un texto con cada vecino y su peso, sacándolos de la lista de tuplas (vecino, peso) que está en vecinos.
- Luego print(f" {vertice} -> {vecinos_str}") imprime una línea con el vértice y todos sus vecinos con pesos.
- Finalmente imprime líneas para separar la lista.

Método bfs(self, inicio)

- Primero verifica con `if inicio not in self.grafo`: si el vértice existe; si no, muestra un mensaje y devuelve lista vacía.
- Crea el conjunto visitados = `set()` para llevar registro de los vértices ya visitados.
- Inicializa la cola `cola = deque([inicio])` con el vértice inicial para procesarlo primero.
- Añade inicio a visitados para marcarlo como visitado.
- Inicia un ciclo `while cola`: que se ejecuta mientras haya vértices en la cola.
- Dentro del ciclo, saca el primer vértice `actual = cola.popleft()` para procesarlo.
- Añade actual a la lista recorrido.
- Para cada vecino de actual (usando `for vecino in self.obtener_vecinos(actual):`), si no está visitado, lo añade a visitados y lo pone al final de la cola para procesarlo después.
- Al terminar el ciclo, devuelve la lista recorrido con el orden en que se visitaron los vértices.

```

def es_conexo(self):
    """
    Devuelve True si el grafo (no dirigido) es conexo.
    Estrategia: Si el BFS desde cualquier vértice recorre todos, entonces es conexo.
    """
    if not self.grafo:
        return True
    inicio = next(iter(self.grafo))
    visitados = set(self.bfs(inicio))
    return len(visitados) == len(self.grafo)

def encontrar_camino(self, inicio, fin):
    """
    Devuelve un camino simple de inicio a fin usando BFS, reconstruyendo con padres.
    Si no hay camino o los vértices no existen, devuelve [].
    """
    if inicio not in self.grafo or fin not in self.grafo:
        return []
    padres = {}
    visitados = set([inicio])
    cola = deque([inicio])
    padres[inicio] = None
    while cola:
        actual = cola.popleft()
        if actual == fin:
            # Reconstruir el camino usando padres
            camino = []
            while actual is not None:
                camino.append(actual)
                actual = padres[actual]
            return list(reversed(camino))
        for vecino in self.obtener_vecinos(actual):
            if vecino not in visitados:
                padres[vecino] = actual
                visitados.add(vecino)
                cola.append(vecino)
    return []

```

es_conexo(self)

- Toma un vértice cualquiera (inicio) con `next(iter(self.grafo))`.
- Usa `bfs(inicio)` para obtener todos los vértices alcanzables desde ese vértice.
- Compara si la cantidad de vértices visitados es igual al total de vértices del grafo.
- Retorna True si son iguales (grafo conexo), False si no.

encontrar_camino(self, inicio, fin)

- Verifica que inicio y fin existan en el grafo.
- Usa BFS para buscar el camino desde inicio hasta fin.

- Lleva un registro de los padres de cada vértice para reconstruir el camino.
- Cuando encuentra fin, reconstruye y devuelve el camino.
- Si no encuentra, devuelve lista vacía.

```
class Funciones:
    def opcion_agregar_vertice(grafo):
        vertice = input("Nombre del vértice: ").strip()
        grafo.agregar_vertice(vertice)
        print(f"Vértice '{vertice}' agregado.")
```

opcion_agregar_vertice(grafo)

- Solicita al usuario un nombre para un vértice.
- Llama a grafo.agregar_vertice() para añadirlo.
- Imprime confirmación de que se agregó.

```
def opcion_agregar_arista(grafo):
    u = input("Vértice de inicio: ").strip()
    v = input("Vértice de destino: ").strip()
    try:
        peso = int(input("Peso (opcional, por defecto 1): ").strip() or "1")
    except ValueError:
        print("Peso inválido, se usará 1.")
        peso = 1
    grafo.agregar_arista(u, v, peso)
    if grafo.es_dirigido:
        print(f"Arista '{u}' -> '{v}' agregada (peso={peso}).")
    else:
        print(f"Arista '{u}' <-> '{v}' agregada (peso={peso}).")
```

opcion_agregar_arista(grafo)

- Pide al usuario los vértices de inicio y destino.
- Pide opcionalmente un peso (por defecto 1).
- Usa grafo.agregar_arista() para crear la arista.
- Informa al usuario si el grafo es dirigido o no y el peso asignado.


```
def opcion_consultar_vecinos(grafo):
    vertice = input("Vértice a consultar: ").strip()
    vecinos = grafo.obtener_vecinos(vertice)
    if vecinos:
        print(f"Vecinos de '{vertice}': {vecinos}")
    else:
        if vertice in grafo.grafo:
            print(f"El vértice '{vertice}' existe pero no tiene vecinos.")
        else:
            print(f"El vértice '{vertice}' NO existe en el grafo.")
```

opcion_consultar_vecinos(grafo)

- Solicita un vértice.
- Llama a grafo.obtener_vecinos() para obtener sus vecinos.
- Muestra los vecinos si existen, o indica si el vértice existe pero no tiene vecinos, o si no existe.

```
def opcion_verificar_arista(grafo):
    u = input("Vértice de inicio: ").strip()
    v = input("Vértice de destino: ").strip()
    if grafo.existe_arista(u, v):
        print(f"Sí existe una arista entre '{u}' y '{v}'.")
    else:
        print(f"No existe una arista entre '{u}' y '{v}'.")
```

opcion_verificar_arista(grafo)

- Pide los vértices inicio y destino.
- Usa grafo.existe_arista() para verificar si existe arista entre ellos.
- Imprime resultado de la verificación.

```
def opcion_mostrar_adyacencia(grafo):
    grafo.mostrar_adyacencia()

def opcion_bfs(grafo):
    inicio = input("Vértice inicial para BFS: ").strip()
    resultado = grafo.bfs(inicio)
    if resultado:
        print(f"Recorrido BFS desde '{inicio}': {resultado}")

def opcion_dfs(grafo):
    inicio = input("Vértice inicial para DFS: ").strip()
    resultado = grafo.dfs(inicio)
    if resultado:
        print(f"Recorrido DFS desde '{inicio}': {resultado}")
```

```

def opcion_desconexo(grafo):
    print("Agregando vértice 'F' (desconexo)...")
    grafo.agregar_vertice('F')
    grafo.mostrar_adyacencia()
    inicio = input("Vértice inicial para BFS/DFS: ").strip()
    print(f"BFS: {grafo.bfs(inicio)}")
    print(f"DFS: {grafo.dfs(inicio)}")

# Métodos nuevos para ejercicio 3
def opcion_es_conexo(grafo):
    if grafo.es_conexo():
        print("El grafo es conexo.")
    else:
        print("El grafo NO es conexo.")

def opcion_encontrar_camino(grafo):
    inicio = input("Vértice de inicio: ").strip()
    fin = input("Vértice destino: ").strip()
    camino = grafo.encontrar_camino(inicio, fin)
    if camino:
        print(f"Camino de '{inicio}' a '{fin}': {camino}")
    else:
        print(f"No existe camino entre '{inicio}' y '{fin}'.")

```

opcion_desconexo(grafo)

- Agrega un vértice desconectado 'F'.
- Muestra la lista de adyacencia.
- Pide un vértice inicial para BFS y DFS.
- Muestra los recorridos desde ese vértice (puede no alcanzar al vértice desconectado)

opcion_es_conexo(grafo)

- Usa grafo.es_conexo() para determinar si el grafo está conectado.
- Imprime si el grafo es conexo o no.

opcion_encontrar_camino(grafo)

- Solicita vértices de inicio y destino.
- Usa grafo.encontrar_camino(inicio, fin) para buscar un camino simple.
- Muestra el camino encontr

opcion_mostrar_adyacencia(grafo)

- Llama a grafo.mostrar_adyacencia() para imprimir la lista de adyacencia completa.

opcion_bfs(grafo)

- Solicita un vértice inicial.
- Ejecuta grafo.bfs() para hacer recorrido en anchura desde ese vértice.
- Muestra el recorrido oado o indica que no existe.

opcion_dfs(grafo)

- Solicita un vértice inicial.
- Ejecuta grafo.dfs() para hacer recorrido en profundidad desde ese vértice.
- Muestra el recorrido obtenido.

main.py

```
def menu():
    # Menú modular validando la entrada y usando match-case
    grafo = Grafo()
    while True:
        print("\n--- MENÚ GRAFOS ---")
        print("1. Agregar vértice")
        print("2. Agregar arista")
        print("3. Consultar vecinos de un vértice")
        print("4. Verificar existencia de arista")
        print("5. Mostrar lista de adyacencia")
        print("6. Recorrido BFS")
        print("7. Recorrido DFS")
        print("8. Prueba de recorrido en grafo desconexo")
        print("9. Verificar si el grafo es conexo")
        print("10. Encontrar camino entre dos vértices")
        print("0. Salir")
        print("-----")
        try:
            opcion = int(input("Elige una opción: "))
        except ValueError:
            print("Por favor, introduce un número válido.")
            continue
        match opcion:
            case 1:
                f.opcion_agregar_vertice(grafo)
                limpiar_pantalla()
            case 2:
                f.opcion_agregar_arista(grafo)
                limpiar_pantalla()
            case 3:
                f.opcion_consultar_vecinos(grafo)
                limpiar_pantalla()
            case 4:
                f.opcion_verificar_arista(grafo)
                limpiar_pantalla()
```

```

    case 5:
        f.opcion_mostrar_adyacencia(grafo)
        limpiar_pantalla()
    case 6:
        f.opcion_bfs(grafo)
        limpiar_pantalla()
    case 7:
        f.opcion_dfs(grafo)
        limpiar_pantalla()
    case 8:
        f.opcion_desconexo(grafo)
        limpiar_pantalla()
    case 9:
        f.opcion_es_conexo(grafo)
        limpiar_pantalla()
    case 10:
        f.opcion_encontrar_camino(grafo)
        limpiar_pantalla()

    case 0:
        print("|Hasta luego!")
        break
    case _:
        print("Opción no válida.")

if __name__ == '__main__':
    menu()

```

Se importan las clases necesarias y se define un menú interactivo para manipular un grafo.

- Crea un objeto Grafo.
- Muestra opciones para agregar vértices y aristas, consultar vecinos, verificar aristas, mostrar la lista de adyacencia, hacer recorridos BFS y DFS, probar grafos desconectados, verificar conectividad y encontrar caminos.
- Cada opción llama a funciones específicas de la clase Funciones.
- Usa un bucle infinito para mostrar el menú hasta que el usuario elige salir (opción 0).
- La pantalla se limpia tras cada acción para mejorar la experiencia visual.